

JavaScript: Rundungsfehler verstehen und vermeiden

1. Beispiel mit Rundungsfehler

In JavaScript werden Dezimalzahlen als Gleitkommazahlen gespeichert. Dadurch lassen sich Werte wie 0.1 und 0.2 intern nicht exakt darstellen. Deshalb ergibt die Addition sichtbar 0.30000000000000004 statt exakt 0.3.

```
let a = 0.1;
let b = 0.2;
console.log(`Ausgabe: ${a + b}`);
console.log(a + b);
```

Ausgabe auf der Webseite:

Ausgabe: 0.30000000000000004
0.30000000000000004

Binäre Darstellung von a, b und dem Resultat a + b (IEEE 754 Double)

Jede Zahl besteht aus **1 Bit Vorzeichen**, **11 Bit Exponent** und **52 Bit Mantisse**. Genau diese interne Darstellung von 0.1 = $1/16 + 1/32 + 1/256 + \dots$ und 0.2 führt dazu, dass die Werte nicht exakt gespeichert werden. Auch das Resultat $a + b$ lässt sich dadurch als leicht ungenauer Wert zeigen. Im Exponenten wurden die 2^{11} Möglichkeiten in der Mitte aufgeteilt, um sowohl positive als auch negative Werte darstellen zu können. Der **Offset** hat deshalb einen Wert von **1023**.

a = 0.1

0 01111111011 10011001100110011001100110011001100110011010

Vorzeichen: 0

Exponent: 01111111011 (dezimal: 1019)

Mantisse: 10011001100110011001100110011001100110011010 (dezimal: 10011001100110011001100110011001100110011010)

Kommazahlen werden dargestellt

durch diese mathematische Formel: $(-1)^{\text{Vorzeichen}} \times 1.\text{Mantisse} \times 2^{(\text{Exponent} - \text{Offset})}$

offset = 1023

Dieses Vorgehen ist besonders sinnvoll bei Geldbeträgen.